

EXPERIMENT 5

Enhancing the TCP Client-Server Model with Multi-Client Support

1. Aim

To extend the previous TCP client-server model to support multiple clients concurrently using multithreading in C.

2. Question

Enhance the TCP Client-Server model to support multiple clients using multithreading.

The server should:

1. Handle multiple clients concurrently using threads.
2. Allow each client to send messages independently.
3. Respond to each client individually.
4. Use proper synchronization wherever necessary.
5. Test the program with multiple clients connecting to the server.

3. Theory

A normal TCP model generally handles one client at a time. To allow multiple clients to communicate with the server simultaneously, multithreading can be used.

A thread is a lightweight unit of execution within a program. whenever a new client connects, the server creates a separate thread to handle that client.

Therefore:

- The main server thread continuously waits for new clients.
- A new thread is created for every connected client.
- Each thread receives the client's message.
- The thread sends a response back to that particular client.
- Multiple clients can communicate with the server independently.

The pthread library is used for creating and managing threads in C.

Since each client is handled by its own thread, the clients can communicate with the server concurrently.

Synchronization

In this implementation, each client thread works independently and does not modify any common data structure. Therefore, no mutex is required for the client-message processing itself. The code uses a mutex around server output so that messages printed by different threads do not interfere with each other.

4. Requirements

- Ubuntu Linux Virtual Machine
- GCC compiler
- C programming language
- TCP sockets
- POSIX Threads (pthread)
- Terminal

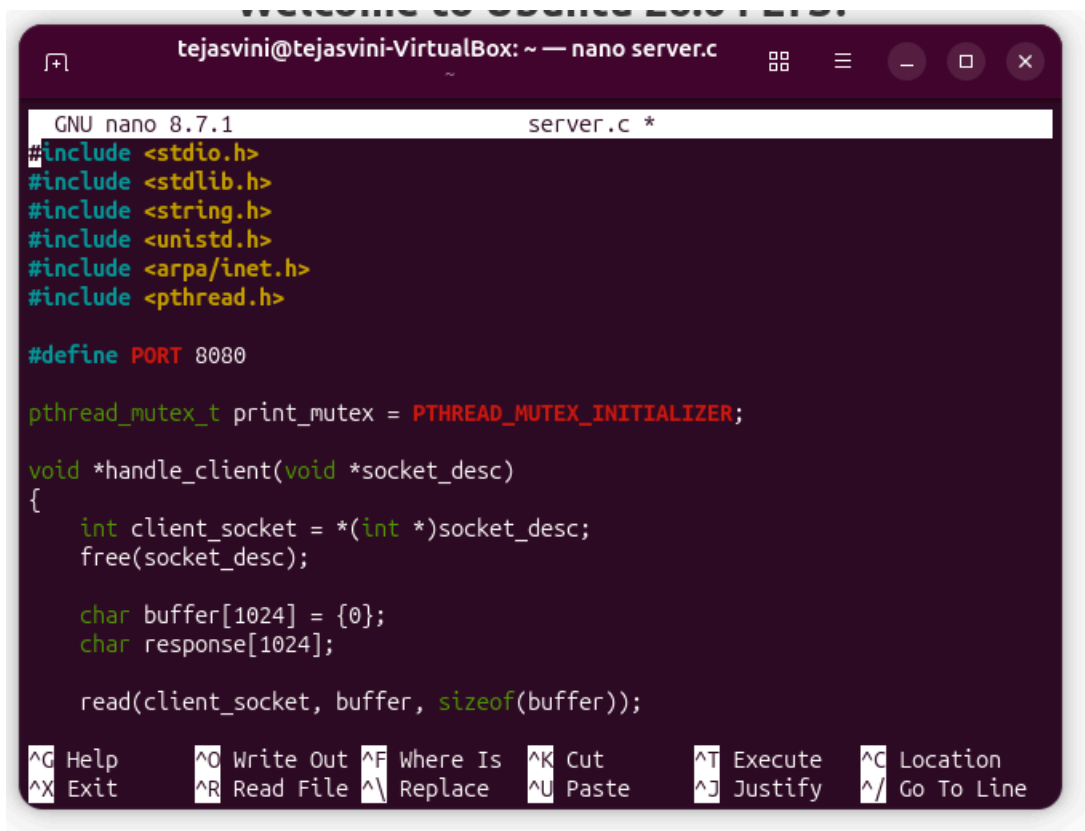
5. Procedure

Step 1: Create the Multi-Client Server

Create a C file named:

server.c

The server creates a new thread whenever a client connects.



```
tejasvini@tejasvini-VirtualBox: ~ — nano server.c
GNU nano 8.7.1 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>

#define PORT 8080

pthread_mutex_t print_mutex = PTHREAD_MUTEX_INITIALIZER;

void *handle_client(void *socket_desc)
{
    int client_socket = *(int *)socket_desc;
    free(socket_desc);

    char buffer[1024] = {0};
    char response[1024];

    read(client_socket, buffer, sizeof(buffer));

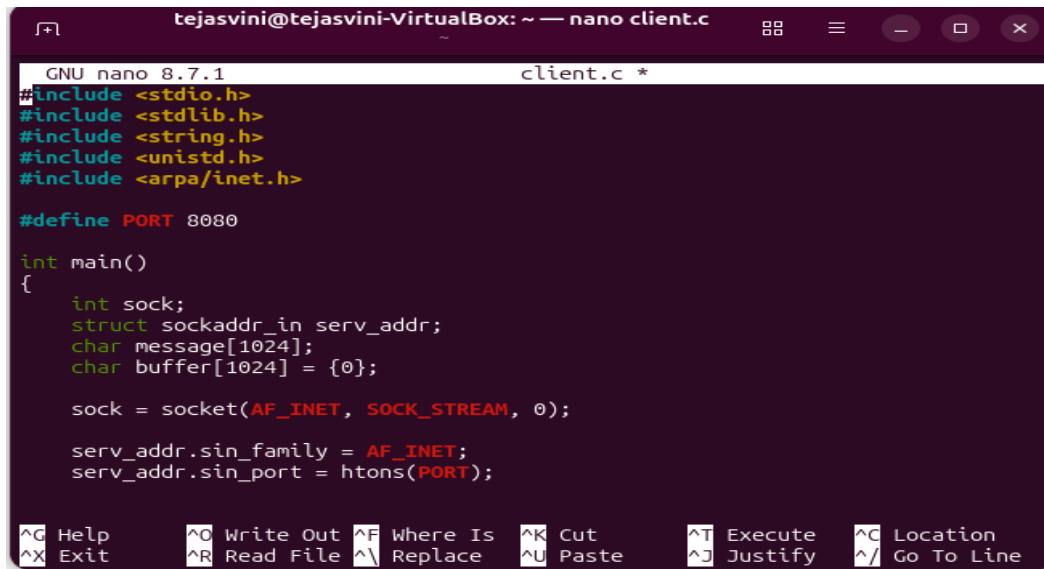
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line
```

Step 2: Create the Client Program

Create a C file named:

client.c

The same client program can be executed multiple times to simulate multiple clients.



```
tejasvini@tejasvini-VirtualBox: ~ — nano client.c
GNU nano 8.7.1 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main()
{
    int sock;
    struct sockaddr_in serv_addr;
    char message[1024];
    char buffer[1024] = {0};

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Step 3: Compile the Server

Compile the server using:

```
gcc server.c -o server -pthread
```

The -pthread option is used because the program uses POSIX threads.

Step 4: Compile the Client

Compile the client using:

```
gcc client.c -o client
```

Step 5: Run the Server

Run the server:

```
./server
```

The server starts listening for incoming client connections.

Step 6: Run Multiple Clients

Open multiple terminals in the same Ubuntu VM and run:

```
./client
```

Run the client program in three terminals to simulate three different clients.

Each client sends its own message to the server.

Step 7: Observe Multi-Client Communication

The server creates a separate thread for every connected client.

Each client receives an individual response from the server.

```
server.c:26:5: note: snprintf output between 51 and 1024 bytes into a destination of size 1024
 26 |     snprintf(response, sizeof(response),
    |     ~~~~~^~~~~~
 27 |     "Server received your message: %s", buffer);
    |     ~~~~~^~~~~~

tejasvini@tejasvini-VirtualBox:~$ ./server
Server is waiting for clients...
Client connected.
Message from Client: Hello from Client
Client connected.
Message from Client: Hello from Client
Client connected.
Message from Client: Hello from Client
```

7. Server Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#define PORT 8080
pthread_mutex_t print_mutex = PTHREAD_MUTEX_INITIALIZER;
void *handle_client(void *socket_desc)
{
    int client_socket = *(int *)socket_desc;
    free(socket_desc);

    char buffer[1024] = {0};
    char response[1024];
    read(client_socket, buffer, sizeof(buffer));
    pthread_mutex_lock(&print_mutex);
    printf("Message from Client: %s\n", buffer);
    pthread_mutex_unlock(&print_mutex);
```

```

    snprintf(response, sizeof(response),
        "Server received your message: %s", buffer);

    send(client_socket, response, strlen(response), 0);
    close(client_socket);
    return NULL;
}

int main()
{
    int server_fd;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 10);
    printf("Server is waiting for clients...\n");
    while (1)
    {
        int *client_socket = malloc(sizeof(int));

        *client_socket = accept(server_fd,
            (struct sockaddr *)&address,
            (socklen_t *)&addrlen);

        printf("Client connected.\n");
        pthread_t thread;
        pthread_create(&thread, NULL, handle_client, client_socket);
    }
}

```

```

        pthread_detach(thread);
    }
    close(server_fd);
    return 0;
}

```

8. Client Program

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080

int main()
{
    int sock;
    struct sockaddr_in serv_addr;
    char message[1024];
    char buffer[1024] = {0};
    sock = socket(AF_INET, SOCK_STREAM, 0);
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
    connect(sock, (struct sockaddr *)&serv_addr,
            sizeof(serv_addr));
    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
    message[strcspn(message, "\n")] = '\0';
    send(sock, message, strlen(message), 0);
    read(sock, buffer, sizeof(buffer));
    printf("Server response: %s\n", buffer);
}

```

```
close(sock);  
return 0;  
}
```

9. Output

Server

Server is waiting for clients...

Client connected.

Message from Client: Hello from Client 1

Client connected.

Message from Client: Hello from Client 2

Client connected.

Message from Client: Hello from Client 3

Client 1

Enter message: Hello from Client 1

Server response: Server received your message: Hello from Client 1

Client 2

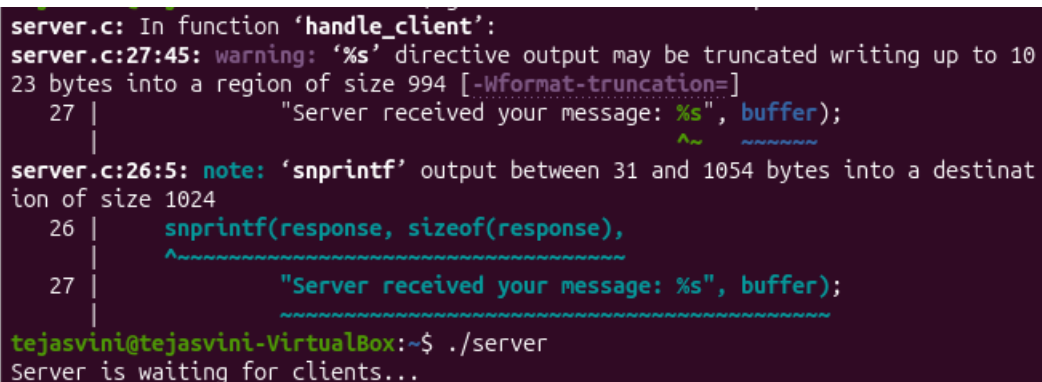
Enter message: Hello from Client 2

Server response: Server received your message: Hello from Client 2

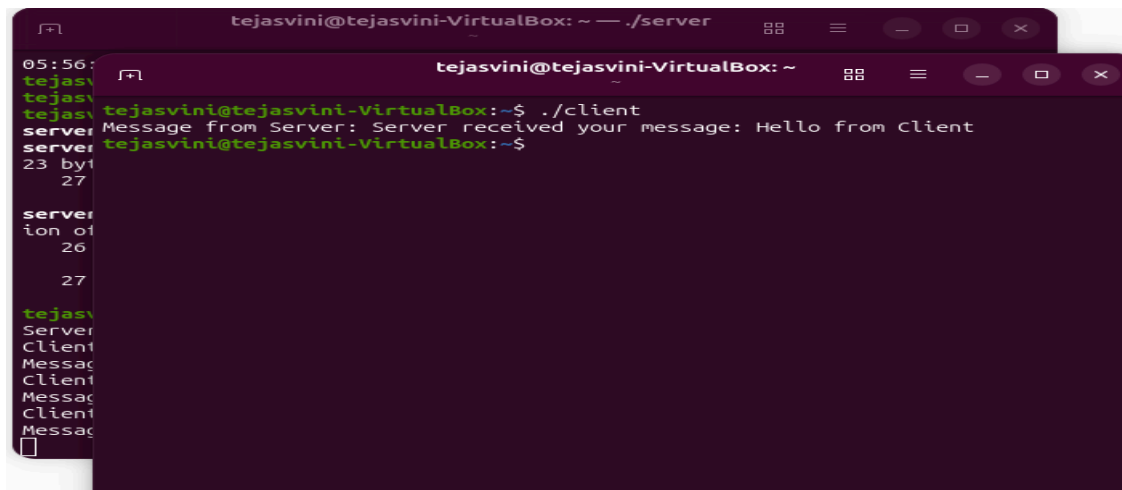
Client 3

Enter message: Hello from Client 3

Server response: Server received your message: Hello from Client 3



```
server.c: In function 'handle_client':  
server.c:27:45: warning: '%s' directive output may be truncated writing up to 10  
23 bytes into a region of size 994 [-Wformat-truncation=]  
27 |         "Server received your message: %s", buffer);  
   |         ~~~~~  
server.c:26:5: note: 'snprintf' output between 31 and 1054 bytes into a destinat  
ion of size 1024  
26 |     snprintf(response, sizeof(response),  
   |     ~~~~~  
27 |     "Server received your message: %s", buffer);  
   |     ~~~~~  
tejasvini@tejasvini-VirtualBox:~$ ./server  
Server is waiting for clients...
```



The image shows two overlapping terminal windows. The background window is titled 'tejasvini@tejasvini-VirtualBox: ~ -- ./server' and shows the following output: 05:56: tejasvini@tejasvini-VirtualBox:~\$./client; Message from Server: Server received your message: Hello from client; tejasvini@tejasvini-VirtualBox:~\$; server: 23 bytes; 27; server: ion of 26; 27; tejasvini@tejasvini-VirtualBox:~\$; Server; Client; Message; Client; Message; Client; Message.



The image shows a terminal window with C code snippets and server output. The code includes: 26 | snprintf(response, sizeof(response),; 27 | "Server received your message: %s", buffer);. Below the code, the server output is shown: tejasvini@tejasvini-VirtualBox:~\$./server; Server is waiting for clients...; Client connected.; Message from Client: Hello from client; Client connected.; Message from Client: Hello from client; Client connected.; Message from Client: Hello from client.

10. Result

The TCP client-server model was successfully extended to support multiple clients using multithreading. The server created separate threads for connected clients, allowing them to communicate independently with the server.

11. Conclusion

Thus, a multi-client TCP server using multithreading in C was successfully implemented and tested. Multiple clients were able to connect to the server concurrently and receive individual responses.